Data Science

Complete Developer Study Notes

Concepts • Practical Examples • Projects • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

Topics Covered (Page 2 - 20)

1. Introduction to Data Science (Overview, Workflow and Use Cases)
2. Python for Data Science (Essential Libraries and Setup)
3. Data Collection and Data Sources (Structured, Unstructured, APIs, Web Scraping)
4. Data Cleaning and Preprocessing (Missing Values, Outliers, Encoding, Scaling)
5. Exploratory Data Analysis (EDA) (Visualization and Insights)
6. Feature Engineering (Feature Creation, Selection and Transformation)
7. Statistical Analysis for Data Science (Descriptive & Inferential Statistics)
8. Probability Distributions and Hypothesis Testing
9. Machine Learning Algorithms (Regression, Classification, Clustering)
10. Model Evaluation and Performance Metrics (Accuracy, Precision, Recall, etc.)
11. Model Tuning and Optimization (Cross Validation, Hyperparameter Tuning)
12. Handling Imbalanced Data (Techniques and Best Practices)
13. Time Series Analysis and Forecasting
14. Natural Language Processing (NLP) Basics (Text Processing and Applications)
15. Real-world Data Science Projects (End-to-End Implementation)
16. Deployment and Productionization (Flask, FastAPI, Docker, Cloud)
17. Best Practices, Common Issues and Troubleshooting
18. Interview Questions (Concepts + Coding Questions)
19. Key Takeaways and Next Steps (Learning Resources and Career Guide)

Rahul Yadav

1. Introduction to Data Science (Overview, Workflow and Use Cases)

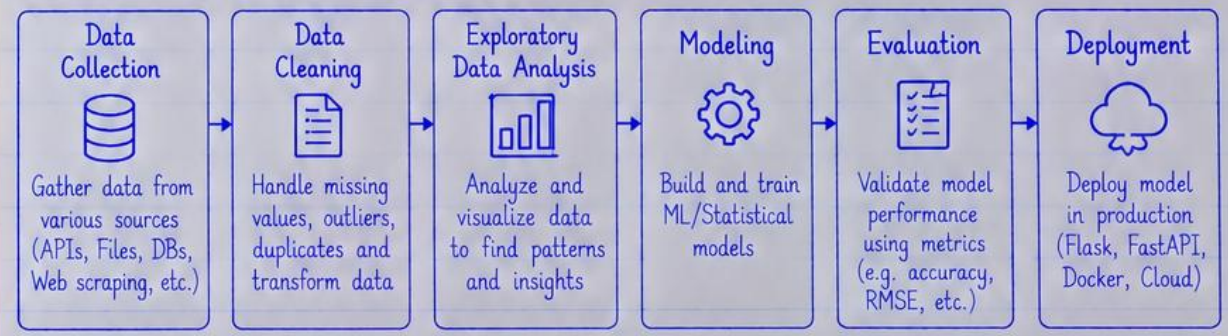
1.1 What is Data Science ?

- Data Science is an interdisciplinary field that uses data, statistics, programming and domain knowledge to extract useful insights from data.
- It combines techniques from Mathematics, Statistics, Machine Learning and Computer Science.
- Goal is to turn raw data into valuable information to make better decisions.
- Data Science is used in almost every industry like finance, healthcare, e-commerce, social media, manufacturing, etc.

1.2 Data Science vs Related Fields

Aspect	Data Science	Machine Learning	Data Analytics	Data Engineering
Focus	End-to-end process	Building predictive models	Analyzing past data for insights	Building data infrastructure
Key Skills	Stats, ML, Python, Domain Knowledge	Algorithms, Modeling	SQL, Visualization, Reporting	ETL, Pipelines, Databases, Big Data
Goal	Insights + Predictions	Learn patterns from data	Understand what happened	Collect, store and process data

1.3 Data Science Workflow



1.4 Popular Applications of Data Science

Industry	Use Cases
Finance	Fraud detection, credit scoring, risk analysis
Healthcare	Disease prediction, medical image analysis
E-commerce	Recommendation systems, customer segmentation
Social Media	Content recommendation, trend analysis
Manufacturing	Predictive maintenance, quality control
Transportation	Demand forecasting, route optimization
Education	Personalized learning, student performance
Agriculture	Crop yield prediction, weather analysis

Why Data Science is Important ?

- Helps in data-driven decision making
- Improves business efficiency and reduces cost
- Enables prediction of future outcomes
- Helps understand customer behavior
- Solves real world complex problems
- High demand career with good opportunities

Key Takeaways

- Data Science combines multiple disciplines
- It follows a structured workflow
- It is widely used across industries
- Foundation topics include Python, Statistics and Machine Learning

Rahul Yadav

2. Python for Data Science (Essential Libraries and Setup)

2.1 Why Python for Data Science ?

- Simple and easy to learn syntax.
- Large ecosystem of powerful libraries.
- Great community support and documentation.
- Widely used in industry and research.
- Works well for data analysis, visualization, machine learning and deep learning.
- Platform independent (Windows, macOS, Linux).

2.2 Important Libraries in Data Science

Library	Purpose	Key Features
NumPy	Numerical computing	N-dimensional arrays, mathematical functions
Pandas	Data manipulation and analysis	DataFrame, Series, read/write files, data cleaning
Matplotlib	Data visualization (basic)	Line, bar, scatter, histogram, etc.
Seaborn	Statistical visualization	Beautiful and informative plots
Scikit-learn	Machine learning	Classification, regression, clustering, preprocessing
TensorFlow	Deep learning	Neural networks, GPU support, production ready
PyTorch	Deep learning	Dynamic computation graph, research friendly
Jupyter Notebook	Interactive development	Run code in cells, visualize results
Others	Utility libraries	Plotly, XGBoost, LightGBM, OpenCV, NLTK, etc.

2.3 Setting up Python Environment

Option 1: Using Anaconda (Recommended)

1. Dewnload Anaconda from official website.
2. Install Anaconda (includes Python + useful libraries).
3. Open Anaconda Prompt.
4. Create a new environment (optional):

```
conda create -n ds python=3.10
```

5. Activate the environment:

```
conda activate ds
```

Option 2: Using pip (Native Python)

1. Install Python from python.org
2. Create a virtual environment:

```
python -m venv ds_env
source ds_env/bin/activate   # Linux/Mac
ds_env\Scripts\activate      # Windows
```

3. Install required libraries:

```
pip install numpy pandas matplotlib
pip install seaborn scikit-learn jupyter
pip install tensorflow  # or pytorch
```

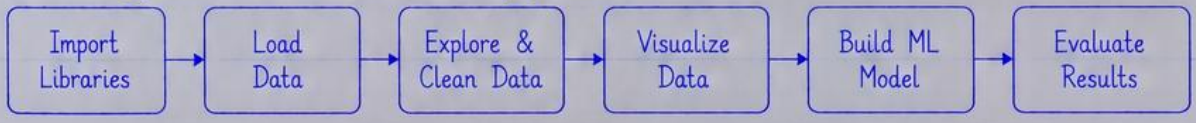
2.4 Verifying Installation

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
print("Libraries imported successfully!")
```

Expected Output

```
Libraries imported successfully!
(No errors)
```

2.5 Typical Workflow in Jupyter Notebook



Rahul Yadav

3. Data Collection and Data Sources (Structured, Unstructured, APIs, Web Scraping)

3.1 What is Data Collection ?

- Data collection is the process of gathering raw data from various sources to solve a business problem or build a machine learning model.
- Quality data is very important for accurate analysis and predictions.
- Data can be collected from internal systems, external sources, APIs, web scraping or real-world devices (Sensors, IoT, etc.).

3.2 Types of Data Sources

Structured Data
- Well-defined format (rows and columns)
- Examples: CSV, Excel, Relational Databases
- Easy to store and analyze

Unstructured Data
- No predefined format
- Examples: Text, Images, Videos, PDF, Audio
- Needs preprocessing to extract useful information

Semi-structured Data
- Partial structure
- Examples: JSON, XML, Logs, Emails
- Flexible and widely used in modern systems

External Data Sources
- APIs (REST, GraphQL)
- Web Scraping
- Public Datasets (Kaggle, UCI, etc.)
- Sensor/IoT Data
- Social Media Data

3.3 Data Collection Workflow

Identify Problem → Find Data Sources → Collect Data → Store Data → Validate & Clean Data → Use for Analysis / ML

3.4 Collecting Data from CSV / Excel

```
import pandas as pd

# Read CSV file
df = pd.read_csv("data.csv")
print(df.head())

# Read Excel file
df_excel = pd.read_excel("data.xlsx")
print(df_excel.head())
```

Common Parameters
- sep : delimiter (default ,)
- header : row number for column names
- usecols : select specific columns
- nrows : read only first n rows
- encoding : file encoding (utf-8, etc.)

3.5 Collecting Data from Databases

```
import pandas as pd
import sqlalchemy

# Create database connection (MySQL example)
engine = sqlalchemy.create_engine(
    "mysql+pymysql://user:password@localhost:3306/dbname"
)

# Read data using SQL query
df = pd.read_sql("SELECT * FROM customers", engine)
print(df.head())
```

Popular Databases
- MySQL
- PostgreSQL
- MongoDB (NoSQL)
- SQLite
- Microsoft SQL Server
- BigQuery
- Amazon Redshift
- Snowflake

Rahul Yadav

4. Data Cleaning and Preprocessing (Missing Values, Outliers, Encoding, Scaling)

4.1 Why Data Cleaning is Important ?

- Real-world data is rarely clean. It can contain missing values, duplicates, outliers, inconsistent formats and noisy data.
- Good data preprocessing improves model performance and accuracy.
- Helps in better analysis, visualization and meaningful insights.
- Garbage in → Garbage out (If data is bad, model will also be bad).

4.2 Common Data Quality Issues

Issue	Description	Example
Missing Values	Some values are not available	NaN, None, empty cells
Duplicates	Repeated records	Same customer multiple times
Inconsistent Data	Different formats for same data	"Male", "male", "M"
Outliers	Abnormally high or low values	Age = 200, Salary = -1000
Wrong Data Types	Incorrect data type	Age stored as string
Noisy Data	Irrelevant or incorrect data	Typos, special characters

4.3 Handling Missing Values

- Remove rows/columns with missing values.
- Fill with mean, median or mode (numerical data).
- Fill with constant value (e.g., 0, "Unknown").
- Use forward fill / backward fill (time series).
- Use model-based imputation (advanced).

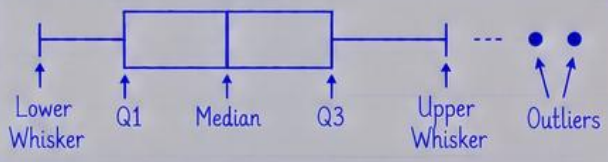
Example (Using Pandas)

```
import pandas as pd
# Fill numeric column with mean
df['age'].fillna(df['age'].mean(), inplace=True)
# Fill categorical column with mode
df['city'].fillna(df['city'].mode()[0], inplace=True)
```

4.4 Handling Outliers

- Outliers are data points that are very different from other values.
- They can affect model performance.
- Methods: IQR, Z-Score, Visualization.

Box Plot (Outliers visualization)



4.5 Feature Encoding (Categorical Variables)

Encoding Method	Description	Use Case	Example
Label Encoding	Convert categories to numbers	Ordinal data (low cardinality)	Male → 0, Female → 1
One-Hot Encoding	Create binary columns	Nominal data	City: Delhi → [1,0,0]
Ordinal Encoding	Custom order to categories	Ordered categories	Low < Medium < High
Target Encoding	Use target mean for encoding	High cardinality	Category → mean(target)

4.6 Feature Scaling (Numerical Variables)

- Scaling brings all features to a similar range.
- Helps algorithms like KNN, SVM, Neural Networks.
- Common methods:

Method	Formula	Use Case
Standardization	$z = (x - \mu) / \sigma$	Data ~ Normal Distribution
Min-Max Scaling	$x' = (x - min) / (max - min)$	Scale to 0 - 1
Robust Scaling	$x' = (x - median) / IQR$	Handle outliers

Rahul Yadav

5. Exploratory Data Analysis (EDA)
(Visualization and Insights)

5.1 What is EDA ?

- Exploratory Data Analysis (EDA) is the process of exploring and analyzing data to understand its main characteristics, patterns, relationships and anomalies.
- Helps in better understanding of data before building machine learning models.
- Uses statistical summaries and visualizations.
- Helps in finding data quality issues and useful insights.

5.2 Goals of EDA

1. Understand the structure and distribution of data.
2. Identify missing values and outliers.
3. Discover patterns and relationships between variables.
4. Generate hypotheses and business insights.
5. Guide data preprocessing and feature engineering.

5.3 Typical EDA Workflow

Load and Understand Data
↓
Check Data Quality
↓
Univariate Analysis
↓
Bivariate / Multivariate Analysis
↓
Draw Insights & Take Action

5.4 Basic Statistical Summary

```
import pandas as pd
df = pd.read_csv("data.csv")

# summary statistics
print(df.describe())

# information about dataset
print(df.info())

# check missing values
print(df.isnull().sum())
```

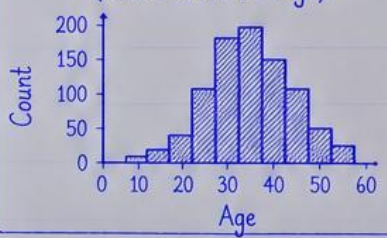
describe() Output (Numeric Columns)

Metric	Age	Salary	Balance
count	1000	1000	1000
mean	35.6	52000	15000
std	10.2	18000	8000
min	18	20000	1000
25%	28	38000	9000
50%	36	50000	14000
75%	42	65000	20000
max	60	120000	50000

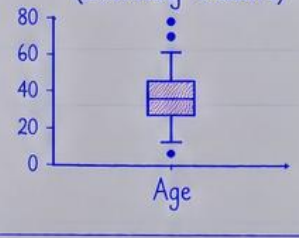
5.5 Univariate Analysis (Single Variable)

- Analyze one variable at a time.
- Understand its distribution, central tendency and spread.
- Useful visualizations: Histogram, Box plot, Count plot.

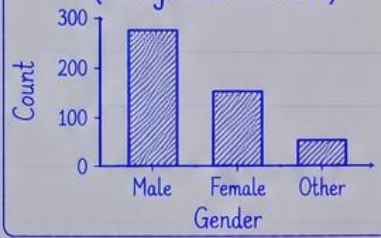
Histogram (Distribution of Age)



Box Plot (Detecting Outliers)



Count Plot (Categorical Variable)



5.6 Bivariate Analysis (Two Variables)

- Analyze relationship between two variables.
- For numeric vs numeric → Scatter plot, correlation.
- For categorical vs numeric → Box plot, Bar plot.
- For categorical vs categorical → Cross tabulation, Heatmap.

Correlation Matrix (Example)

```
import seaborn as sns
import matplotlib.pyplot as plt
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap="Blues")
plt.show()
```

Rahul Yadav

6. Data Visualization in Data Science (Matplotlib, Seaborn, Plotly)

6.1 Why Data Visualization ?

- Helps to understand data easily.
- Find patterns, trends and outliers.
- Makes complex data simple and interactive.
- Useful for analysis, reporting and presentations.
- Essential step in Exploratory Data Analysis (EDA).

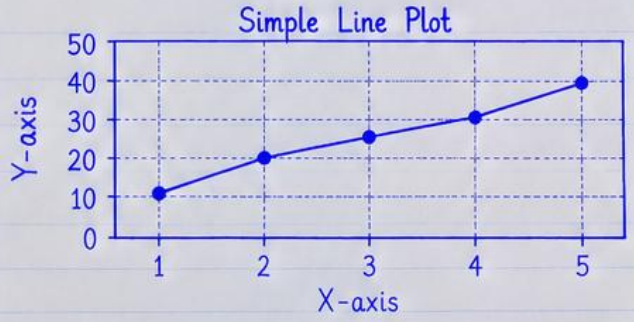
6.2 Popular Visualization Libraries

Library	Purpose / Features
Matplotlib	Basic plots, highly customizable
Seaborn	Statistical plots, beautiful style
Plotly	Interactive plots, web based
Pandas Plot	Quick plotting using pandas
Others	Bokeh, Altair, Dash, etc.

6.3 Basic Plot using Matplotlib

```
import matplotlib.pyplot as plt
# sample data
x = [1, 2, 3, 4, 5]
y = [10, 20, 25, 30, 40]
# line plot
plt.plot(x, y, marker='o')
plt.title('Simple Line Plot')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.grid(True)
plt.show()
```

Output



6.4 Common Plots in Matplotlib

Plot Type	Function	Use Case
Line Plot	plt.plot()	Trend over time
Scatter Plot	plt.scatter()	Relationship between variables
Bar Plot	plt.bar()	Compare categories
Histogram	plt.hist()	Distribution of a variable
Box Plot	plt.boxplot()	Detect outliers
Pie Chart	plt.pie()	Show percentage distribution

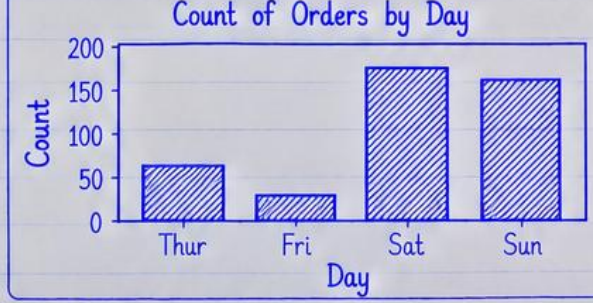
Example: Scatter Plot

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [5, 7, 9, 14, 20]
plt.scatter(x, y, color="blue")
plt.title("Scatter Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

6.5 Visualization using Seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt
# load sample dataset
df = sns.load_dataset("tips")
# count plot
sns.countplot(x="day", data=df)
plt.title('Count of Orders by Day')
plt.xlabel("Day")
plt.ylabel("Count")
plt.show()
```

Output



6.6 Advanced Visualization using Plotly (Interactive)

```
import plotly.express as px
# sample data
df = px.data.iris()
fig = px.scatter(df, x="sepal_width", y="sepal_length",
      color="species", title="Iris Scatter Plot")
fig.show()
```

Key Takeaways

- Use Matplotlib for basic and customized plots.
- Use Seaborn for statistical and beautiful plots.
- Use Plotly for interactive visualizations.
- Choose the right plot based on your data and goal.
- Good visualizations help in better decision making.

Rahul Yadav

7. Data Visualization with Plotly (Advanced)

7.1 Why Use Plotly for Advanced Visualization?

- Interactive and dynamic visualizations.
- Supports 2D, 3D, and web-based plots.
- Highly customizable with layouts, themes and styles.
- Useful for dashboards and real-time data.
- Great for data exploration and presentation.
- Works well with pandas, numpy and other libraries.

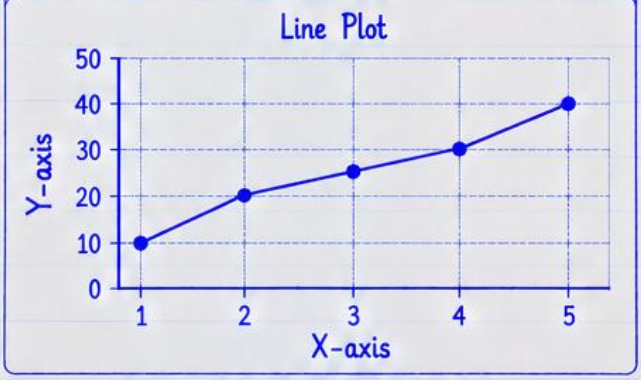
7.2 Plotly vs Matplotlib / Seaborn

Feature	Plotly	Matplotlib	Seaborn
Interactivity	Yes	No	Limited
Customization	High	High	Medium
3D Plots	Yes	Limited	No
Web-based	Yes	No	No
Ease of Use	Medium	Medium	Easy
Best Use Case	Interactive Dashboard	Static Plots	Statistical Plots

7.3 Basic Plotly Line Plot

```
import plotly.express as px
# sample data
x = [1, 2, 3, 4, 5]
y = [10, 20, 25, 30, 40]
fig = px.line(x = x, y = y,
    markers = True, title = "Line Plot")
fig.update_layout(
    xaxis_title = "X-axis",
    yaxis_title = "Y-axis")
fig.show()
```

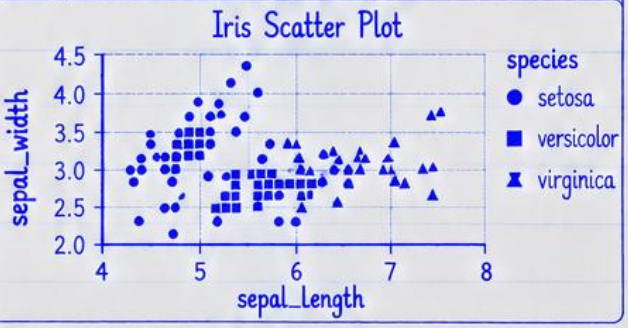
Output : Interactive Line Plot



7.4 Scatter Plot with Customization

```
import plotly.express as px
# sample data
df = px.data.iris()
fig = px.scatter(df, x = "sepal_length",
    y = "sepal_width", color = "species",
    title = "Iris Scatter Plot",
    size = "petal_Length")
fig.show()
```

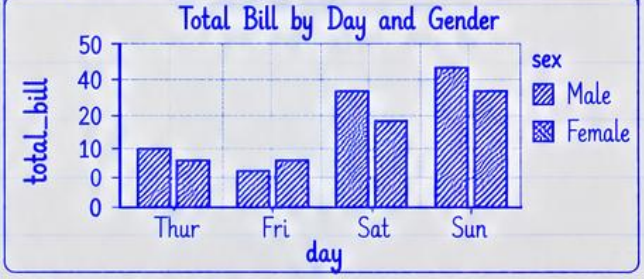
Output : Interactive Scatter Plot



7.5 Bar Plot with Categories

```
import plotly.express as px
df = px.data.tips()
fig = px.bar(df, x = "day", y = "total_bill",
    color = "sex", barmode = "group",
    title = "Total Bill by Day and Gender")
fig.show()
```

Output : Bar Plot



7.6 Advanced Visualizations with Plotly

- 3D Plots : Scatter3D, Surface, Mesh
- Subplots : Combine multiple plots in one figure
- Geographical Plots : Choropleth maps
- Animations : Visualize data over time
- Dashboards : Build interactive dashboards with Plotly
- Custom Themes : Change colors, fonts, layout
- Export : Save as HTML, PNG, or embed in web apps

7.7 Key Takeaways

- Plotly provides interactive and beautiful visualizations.
- Supports a wide range of plot types (2D, 3D, maps, etc.).
- Can be easily integrated with pandas and other libraries.
- Useful for exploratory data analysis and dashboards.
- Choose the right plot type based on your data and goal.
- Interactive plots help in better data understanding and communication.

Rahul Yadav

8. Dashboard Development with Plotly (Dash)

8.1 What is Plotly Dash ?

- Dash is a Python framework by Plotly for building interactive web dashboards.
- Uses Plotly for visualizations.
- Supports real-time data, callbacks, and interactive components.
- Easy to build and deploy analytical dashboards.
- Commonly used in data science, analytics and business intelligence applications).

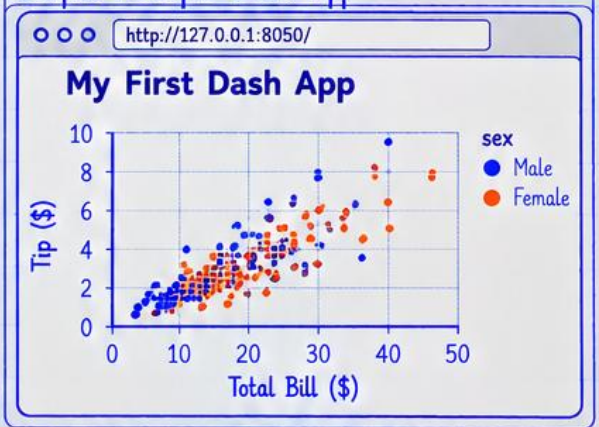
8.2 Key Features of Dash

- Build interactive web dashboards using Python.
- Wide range of UI components (graphs, dropdowns, sliders, tables, etc.).
- Callback system for interactivity.
- Supports real-time data updates.
- Highly customizable layout and styling.
- Can be deployed on cloud (Heroku, AWS, etc.).
- Useful for data-driven decision making.

8.3 Basic Dash App Structure

```
import dash
from dash import dcc, html
import plotly.express as px
df = px.data.tips()
app = dash.Dash(__name__)
app.layout = html.Div([
    html.H1("My First Dash App"),
    dcc.Graph(
        figure = px.scatter(df, x = "total_bill",
                 y = "tip", color = "sex")
    )
])
if __name__ == "__main__":
    app.run_server(debug=True)
```

Output : Simple Dash App



8.4 Important Dash Components

Component	Purpose	Example
html.Div	Container for layout	html.Div([])
html.H1, H2, H3	Headings	html.H1("Title")
dcc.Graph	Interactive plots	dcc.Graph(figure=fig)
dcc.Dropdown	Dropdown selection	dcc.Dropdown(options=[])
dcc.Slider	Slider input	dcc.Slider(min=0, max=10)
dcc.Input	Text/number input	dcc.Input(type='text')
dcc.DatePickerRange	Date range selector	dcc.DatePickerRange()
dcc.Tabs	Multiple tabs	dcc.Tabs([])
dash_table.DataTable	Display tabular data	dash_table.DataTable()

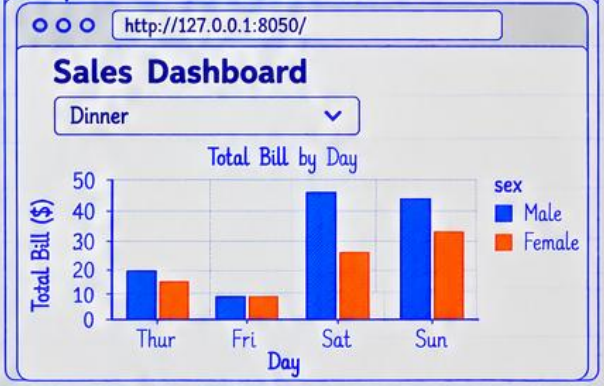
8.5 Layout with Multiple Components

```
app.layout = html.Div([
    html.H1("Sales Dashboard"),
    html.Div([
        dcc.Dropdown(
            id='category-dropdown',
            options=[{'label': 'All', 'value': 'all'},
                     {'label': 'Lunch', 'value': 'lunch'},
                     {'label': 'Dinner', 'value': 'dinner'}],
            value='all'
        ),
        dcc.Graph(id='sales-plot'),
    ], style={'width': '80%', 'margin': 'auto'})
])
```

8.6 Adding Interactivity with Callbacks

```
@app.callback(
    Output('sales-plot', 'figure'),
    Input('category-dropdown', 'value')
)
def update_graph(selected_category):
    if selected_category == 'all':
        dff = df
    else:
        dff = df[df['time'] == selected_category]
    fig = px.bar(dff, x='day', y='total_bill', color='sex',
          barmode='group', title='Total Bill by Day')
    return fig
```

Output : Interactive Dashboard



8.7 Deployment Options

Platform	Command / Method	Use Case
Local	python app.py	Development / Testing
Heroku	Procfile + git deploy	Small projects
Render	Connect GitHub repo	Free hosting
AWS (EC2)	Run with Gunicorn	Production
Docker	Dockerfile + container	Portable deployment

8.8 Key Takeaways

- Plotly Dash makes it easy to build interactive dashboards.
- Use Dash components for rich user interface.
- Callbacks enable dynamic and real-time updates.
- Can be used for business analytics, monitoring, and reporting.
- Supports deployment on multiple platforms.
- Highly customizable and beginner-friendly.

Rahul Yadav

9. Machine Learning Algorithms (Regression, Classification, Clustering)

9.1 What is Machine Learning ?

- Machine Learning (ML) is a subset of AI that learns patterns from data without being explicitly programmed.
- It uses statistical techniques to make predictions or decisions.
- Learns from past data (training) and generalizes to unseen data (testing).
- Widely used in recommendation systems, fraud detection, image recognition, NLP, etc.

9.2 Types of Machine Learning

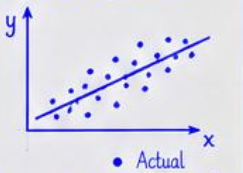
1. **Supervised Learning**
   - Learns from labeled data (input → output).
   - Used for regression and classification.
   - Examples: House price prediction, spam detection.
2. **Unsupervised Learning**
   - Learns patterns from unlabeled data.
   - Used for clustering, dimensionality reduction.
   - Examples: Customer segmentation, topic modeling.
3. **Reinforcement Learning**
   - Learn through trial and error with rewards.
   - Used in robotics, game playing, recommendation.
   - Example: Self-driving cars, AlphaGo.

9.3 Regression Algorithms

- Regression is used to predict a continuous numerical value.
- Examples: House price, sales amount, temperature.
- Common algorithms:
  1. Linear Regression
  2. Polynomial Regression
  3. Decision Tree Regression
  4. Random Forest Regression
  5. Support Vector Regression (SVR)

9.4 Linear Regression

- Models the relationship between input features (X) and a continuous target (y) using a straight line.
- Equation : $y = \theta_0 + \theta_1 x$
  - θ_0 = intercept (bias)
  - θ_1 = slope (coefficient)
  - Learned by minimizing the Mean Squared Error (MSE).



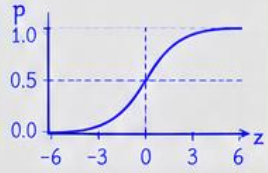
Cost Function : $J(\theta) = \frac{1}{m} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2$

9.5 Classification Algorithms

- Classification is used to predict a discrete class label or category.
- Examples: Spam or Not Spam, Disease or Not, Fraud or Not.
- Common algorithms:
  1. Logistic Regression
  2. Decision Tree Classifier
  3. Random Forest Classifier
  4. Support Vector Machine (SVM)
  5. K-Nearest Neighbors (KNN)
  6. Naive Bayes
  7. Gradient Boosting (XGBoost, LightGBM, etc.)

9.6 Logistic Regression (Binary Classification)

- Predicts the probability that an input belongs to a class (0 or 1).
- Uses sigmoid function to convert output to probability.
- Equation : $p = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x)}}$
- p = probability (0 to 1)
- If $p \geq 0.5$ → class 1
- If $p < 0.5$ → class 0

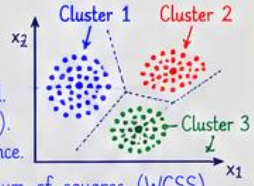


9.7 Clustering Algorithms

- Clustering is an unsupervised learning technique to group similar data points together.
- Used for customer segmentation, image grouping, anomaly detection, etc.
- Common algorithms:
  1. K-Means Clustering
  2. Hierarchical Clustering
  3. DBSCAN (Density Based)
  4. Gaussian Mixture Model (GMM)
  5. Mean Shift

9.8 K-Means Clustering

- Divides data into K clusters based on similarity.
- Steps:
  1. Choose number of clusters (K).
  2. Initialize K centroids randomly.
  3. Assign each point to nearest centroid.
  4. Recalculate centroids (mean of points).
  5. Repeat steps 3 and 4 until convergence.



Objective : Minimize within-cluster sum of squares (WCSS)

$$WCSS = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2$$

9.9 Model Evaluation Metrics

Task	Metric	Formula / Description
Regression	MAE	$\frac{1}{m} \sum \lvert y_i - \hat{y}_i \rvert$
	MSE	$\frac{1}{m} \sum (y_i - \hat{y}_i)^2$
	RMSE	$\sqrt{MSE}$
	R^2 Score	$1 - \frac{SS_{res}}{SS_{tot}}$
Classification	Accuracy	(TP + TN) / (TP + TN + FP + FN)
	Precision	TP / (TP + FP)
	Recall	TP / (TP + FN)
	F1-Score	$2 \times \left(\frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}\right)$

9.10 Key Takeaways

1. Regression predicts continuous values, classification predicts categorical labels, and clustering groups similar data.
2. Choose the right algorithm based on the problem and data.
3. Preprocess data properly for better performance.
4. Evaluate models using appropriate metrics.
5. Tune hyperparameters to improve results.
6. Use ensemble methods (Random Forest, XGBoost) for better accuracy.
7. Apply ML in real-world problems like recommendation systems, fraud detection, customer segmentation, etc.

Rahul Yadav

10. Model Evaluation and Performance Metrics (Accuracy, Precision, Recall, etc.)

10.1 Why Model Evaluation is Important ?

- Helps us understand how well the model performs on unseen data.
- Allows comparison between different models.
- Helps in choosing the right model for the problem.
- Prevents overfitting and underfitting.
- Uses different metrics for different types of problems (classification, regression, clustering).

10.2 Types of Model Evaluation

1. **Classification Metrics**
   - Accuracy, Precision, Recall, F1-Score, ROC-AUC
2. **Regression Metrics**
   - MAE, MSE, RMSE, R^2 Score
3. **Clustering Metrics**
   - Silhouette Score, Davies-Bouldin Index
4. **Model Validation Techniques**
   - Train-Test Split, Cross Validation, Holdout

10.3 Confusion Matrix (Classification)

		Predicted Class	
		Positive (1)	Negative (0)
Actual Class	Positive (1)	True Positive (TP)	False Negative (FN)
	Negative (0)	False Positive (FP)	True Negative (TN)

- TP: Correctly predicted positive
- TN: Correctly predicted negative
- FP: Incorrectly predicted positive (Type I error)
- FN: Incorrectly predicted negative (Type II error)

10.4 Classification Evaluation Metrics

1. **Accuracy**
   - Proportion of correct predictions.

$$\text{Accuracy} = \frac{(TP + TN)}{(TP + TN + FP + FN)}$$

2. **Precision**
   - Out of predicted positives, how many are actual positive.

$$\text{Precision} = \frac{TP}{(TP + FP)}$$

3. **Recall (Sensitivity)**
   - Out of actual positives, how many are correctly predicted.

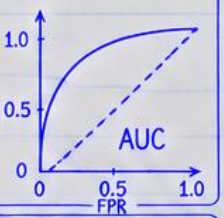
$$\text{Recall} = \frac{TP}{(TP + FN)}$$

4. **F1-Score**
   - Harmonic mean of Precision and Recall.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

10.5 ROC Curve and AUC

- ROC (Receiver Operating Characteristic) curve shows the trade-off between True Positive Rate (TPR) and False Positive Rate (FPR) at different thresholds.
- AUC (Area Under Curve) measures the model's ability to distinguish between classes.
- AUC value range: 0 to 1
  - 1.0 → Perfect model
  - 0.5 → Random model
  - < 0.5 → Poor model



10.6 Regression Evaluation Metrics

1. **Mean Absolute Error (MAE)**
   - Average of absolute differences between actual and predicted values.

$$MAE = \frac{1}{n} \sum_{i=1}^{m} |y_i - \hat{y}_i|$$

2. **Mean Squared Error (MSE)**
   - Average of squared differences.

$$MSE = \frac{1}{n} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2$$

3. **Root Mean Squared Error (RMSE)**
   - Square root of MSE.

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{n} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2}$$

4. **R^2 Score (Coefficient of Determination)**
   - Indicates how well the model explains the variance.

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

10.7 Cross Validation (Model Evaluation)

- Technique to evaluate model performance more reliably.
- Data is split into K folds (e.g., K=5 or 10).
- Model is trained K times, each time using a different fold as the validation set.
- Final performance is the average of all folds.

Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Val	Train	Train	Train	Train
...				
Train	Train	Train	Train	Val

10.8 Common Evaluation Metrics Summary

Metric	Used For	Formula / Notes
Accuracy	Classification	$(TP + TN) / (TP + TN + FP + FN)$
Precision	Classification	$TP / (TP + FP)$
Recall	Classification	$TP / (TP + FN)$
F1-Score	Classification	$2 \times P \times R / (P + R)$
ROC-AUC	Classification	Area under ROC curve
MAE	Regression	$\frac{1}{m} \sum \lvert y_i - \hat{y}_i \rvert$
MSE	Regression	$\frac{1}{m} \sum (y_i - \hat{y}_i)^2$
RMSE	Regression	$\sqrt{MSE}$
R^2 Score	Regression	$1 - \sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2$
Silhouette Score	Clustering	Measures cluster separation

10.9 Key Takeaways

1. Use appropriate metrics based on the problem type.
2. Confusion matrix helps to understand model performance.
3. Accuracy alone is not sufficient for imbalanced data.
4. Cross-validation gives a more reliable evaluation.
5. Compare multiple metrics to choose the best model.
6. Visualize results using ROC curve, learning curves, etc.
7. Always test on unseen data (test set) for final evaluation.

Rahul Yadav

11. Model Tuning and Optimization
(Cross Validation, Hyperparameter Tuning)

11.1 What is Model Tuning ?

- Model tuning means finding the best configuration for a model to improve its performance.
- We adjust model parameters (hyperparameters) to get better accuracy, reduce overfitting and improve generalization.
- Good tuning helps the model work well on unseen data.
- Common techniques: Cross Validation, Grid Search, Random Search, Bayesian Optimization, etc.

11.2 Model Tuning vs Model Training

Model Training	Model Tuning
Learns patterns from data	Finds best hyperparameters
Uses fixed parameters	Tries multiple parameter combinations
Optimizes model weights	Optimizes model performance
Done once	Done multiple times (iterative)
Output: trained model	Output: best model with optimal parameters

11.3 What are Hyperparameters ?

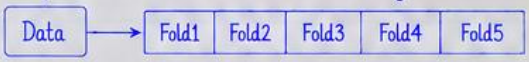
- Hyperparameters are configuration settings that are set before training the model.
- They control the learning process and significantly impact model performance.
- Examples :
  - Learning rate (for gradient-based models)
  - Number of trees (Random Forest)
  - Regularization parameter (C, alpha)
  - Number of clusters (K-Means)
  - Max depth (Decision Tree)
  - Batch size, Epochs (Neural Networks)

11.4 Types of Hyperparameter Tuning

1. **Grid Search**
   - Tries all possible combinations from a given parameter grid.
   - Simple but computationally expensive.
2. **Random Search**
   - Randomly samples combinations from the parameter space.
   - Faster than Grid Search and often gives good results.
3. **Bayesian Optimization**
   - Uses past results to choose the next best set of parameters.
   - More efficient and works well for complex models.
4. **Manual Tuning**
   - Based on experience and domain knowledge.
   - Useful for small parameter space.

11.5 K-Fold Cross Validation

- Data is split into K equal parts (folds).
- Model is trained K times, each time using K-1 folds for training and 1 fold for validation.
- Final performance is the average of all folds.
- Helps to get a reliable estimate of model performance and reduces overfitting.



Iteration 1	Val	Train	Train	Train	Train
Iteration 2	Train	Val	Train	Train	Train
Iteration 3	Train	Train	Val	Train	Train
Iteration 4	Train	Train	Train	Val	Train
Iteration 5	Train	Train	Train	Train	Val

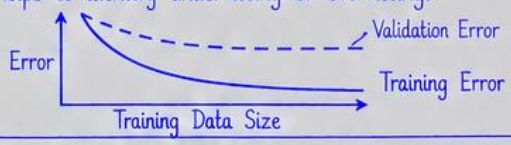
11.6 Train / Validation / Test Split

- Train Set : Used to train the model.
- Validation Set : Used to tune hyperparameters and select the best model.
- Test Set : Used to evaluate final performance on unseen data.



11.7 Learning Curves

- Learning curves show how model performance changes with the size of training data.
- Helps to identify underfitting or overfitting.



11.8 Regularization Techniques

- Regularization helps to prevent overfitting by adding a penalty to the model complexity.
- Common types :
  1. L1 Regularization (Lasso) → Adds $|w|$ penalty
  2. L2 Regularization (Ridge) → Adds w^2 penalty
  3. Elastic Net → Combination of L1 and L2
  4. Dropout → Used in Neural Networks

11.9 Key Takeaways

1. Hyperparameters control the learning process.
2. Use cross validation to get a reliable performance estimate.
3. Grid Search, Random Search and Bayesian Optimization are common tuning techniques.
4. Proper model tuning improves accuracy and generalization.
5. Use learning curves to diagnose underfitting or overfitting.
6. Regularization helps to prevent overfitting.
7. Always evaluate the final model on a separate test set.

Rahul Yadav

12. Handling Imbalanced Data
(Techniques and Best Practices)

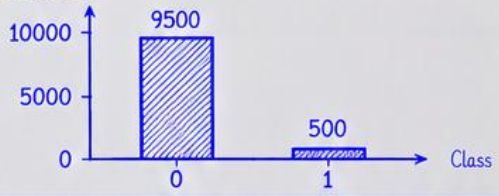
12.1 What is Imbalanced Data ?

- Imbalanced data occurs when the number of samples in one class is significantly higher than the other class(es).
- Common in real-world problems like fraud detection, disease prediction, churn prediction, spam detection, etc.
- Causes models to be biased towards the majority class and perform poorly on the minority class.
- Need special techniques to handle imbalance.

12.2 Example of Imbalanced Data

Class	Count	Percentage
Normal (0)	9500	95%
Fraud (1)	500	5%

- Visualization :



12.3 Problems with Imbalanced Data

- Model tends to predict the majority class always.
- High overall accuracy but poor minority class performance.
- Metrics like accuracy can be misleading.
- Important rare events (e.g., fraud) may be missed.
- Causes business and real-world impact.

12.4 Evaluation Metrics for Imbalanced Data

- Accuracy is not a good metric. Use the following :

Metric	Description
Precision	Out of predicted positives, how many are correct.
Recall (Sensitivity)	Out of actual positives, how many are detected.
F1-Score	Harmonic mean of Precision and Recall.
ROC-AUC	Measures separability between classes.
PR-AUC	More useful for imbalanced data.
Confusion Matrix	Gives complete picture of predictions.

12.5 Confusion Matrix (Minority Class Focus)

Actual \ Predicted	Negative (0)	Positive (1)
Negative (0)	TN	FP
Positive (1)	FN	TP

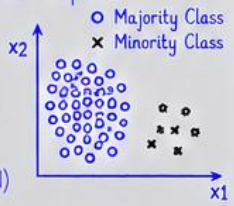
- In imbalanced data, focus more on :
  - True Positives (TP) → correctly detected minority class
  - False Negatives (FN) → missed minority class

12.6 Techniques to Handle Imbalanced Data

1. **Data-level Techniques** (Resampling)
   - Undersampling (Random, Tomek Links, ENN)
   - Oversampling (Random, SMOTE, ADASYN)
   - Hybrid Methods (SMOTE + Tomek Links)
2. **Algorithm-level Techniques**
   - Class Weighting (e.g., class_weight in sklearn)
   - Cost-Sensitive Learning
   - Ensemble Methods (Balanced Random Forest, EasyEnsemble)
3. **Evaluation-level Techniques**
   - Use appropriate metrics (F1, PR-AUC, Recall)
   - Stratified Sampling in cross-validation.

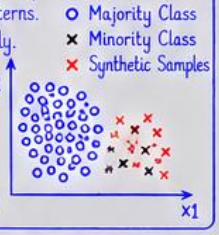
12.7 Undersampling Techniques

- Reduces the number of majority class samples.
- Simpler and faster.
- May lose important information.
- Common Methods :
  1. Random Undersampling
  2. Tomek Links
  3. Edited Nearest Neighbors (ENN)



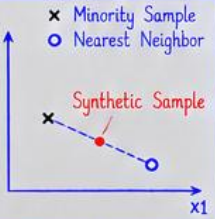
12.8 Oversampling Techniques

- Increases the number of minority class samples.
- Helps the model learn minority class patterns.
- May lead to overfitting if used incorrectly.
- Common Methods :
  1. Random Oversampling
  2. SMOTE (Synthetic Minority (Over-sampling Technique)
  3. ADASYN (Adaptive Synthetic Sampling)



12.9 SMOTE (Working Principle)

- Creates synthetic samples for minority class using nearest neighbors.
- Steps :
  1. For each minority sample, find k nearest neighbors.
  2. Randomly select a neighbor.
  3. Create a new synthetic sample on the line between them.
  4. Repeat until the desired number of samples is reached.



12.10 Algorithm-level Techniques

- **Class Weighting** : Give higher weight to minority class (e.g., class_weight='balanced' in sklearn).
- **Cost-Sensitive Learning** : Assign higher misclassification cost to minority class.
- **Ensemble Methods** :
  - Balanced Random Forest
  - EasyEnsemble
  - AdaBoost with class weights
  - XGBoost with scale_pos_weight

12.11 Best Practices

1. Analyze data distribution before modeling.
2. Use stratified train-test split.
3. Try multiple techniques and compare results.
4. Use appropriate evaluation metrics (F1, Recall, PR-AUC).
5. Combine techniques (e.g., SMOTE + Class Weighting).
6. Perform feature engineering to improve separability.
7. Validate on real-world data and monitor performance.

12.12 Key Takeaways

1. Imbalanced data is common in real-world applications.
2. Accuracy alone is misleading.
3. Use resampling, algorithm-level, and evaluation-level techniques.
4. SMOTE is widely used for oversampling.
5. Evaluate using F1-Score, Recall, and PR-AUC.
6. Proper handling improves model performance and helps detect important rare events.

Rahul Yadav

13. Time Series Analysis and Forecasting

13.1 What is Time Series Analysis ?

- Time series data is a sequence of data points recorded at regular time intervals (daily, monthly, Yearly, etc.).
- Used to analyze past data to understand patterns, trends, and seasonality and to forecast future values.
- Common in business, finance, demand forecasting, weather prediction, stock market, etc.
- Unlike traditional data, time series data is order-dependent (time matters).

13.2 Components of Time Series

- A time series is typically made up of four components:
  1. Trend – long-term movement (increasing or decreasing)
  2. Seasonality – repeating pattern at fixed intervals (e.g., monthly, yearly)
  3. Cyclicity – long-term fluctuations (not fixed period)
  4. Noise/Irregular – random variations

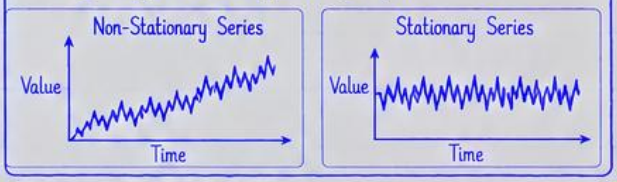


13.3 Types of Time Series

1. Univariate Time Series
   – Only one variable (e.g., daily sales).
2. Multivariate Time Series
   – Multiple variables (e.g., sales, price, temperature).
3. Stationary Time Series
   – Mean, variance and autocorrelation are constant over time.
4. Non-Stationary Time Series
   – Mean, variance or autocorrelation changes over time.

13.4 Stationarity and Tests

- A stationary time series has constant mean, variance and autocorrelation over time.
- Most forecasting models (like ARIMA) require stationary data.
- We can check stationarity using:
  1. Visual inspection (plot)
  2. Rolling mean and rolling std
  3. Augmented Dickey-Fuller (ADF) test



13.5 Common Time Series Models

Model	Description
Naive Model	Uses last observed value as forecast
Moving Average (MA)	Uses average of past values
Autoregressive (AR)	Uses past values to predict next value
ARIMA	Combines AR, MA and differencing (I)
SARIMA	ARIMA with seasonality
Exponential Smoothing	Assigns exponentially decreasing weights
Prophet (Facebook)	Handles trend, seasonality and holidays
LSTM (Deep Learning)	Good for complex patterns and large data

13.6 ARIMA Model

- ARIMA stands for AutoRegressive Integrated Moving Average.
- Parameters : (p, d, q)
  - p : number of autoregressive terms (AR)
  - d : number of differencing (to make data stationary)
  - q : number of moving average terms (MA)

$$ARIMA(p, d, q) = AR(p) + I(d) + MA(q)$$

- Basic Workflow :
  1. Check stationarity (ADF test)
  2. If non-stationary, apply differencing (d)
  3. Use ACF and PACF plots to find p and q
  4. Train ARIMA model and forecast

13.7 SARIMA Model (Seasonal ARIMA)

- Used when the data has seasonal patterns.
- Adds seasonal parameters (P, D, Q, s).
- Formula : SARIMA(p, d, q) × (P, D, Q, s)
  - P : seasonal autoregressive term
  - D : seasonal differencing
  - Q : seasonal moving average term
  - s : seasonal period (e.g., 12 for monthly data)

13.8 Exponential Smoothing

- Gives more weight to recent observations.
- Common Methods :
  1. Simple Exponential Smoothing (no trend, no seasonality)
  2. Holt's Linear Trend (with trend)
  3. Holt-Winters (with trend and seasonality)

$$S_t = \alpha \times Y_t + (1 - \alpha) \times S_{t-1}$$

- α : smoothing factor $(0 < \alpha < 1)$
- Easy to implement and works well for short term forecasts.

13.9 Model Evaluation Metrics

Metric	Formula	Description
MAE	$\frac{1}{m}\sum_{i=1}^{m}\lvert y_i - \hat{y}_i\rvert$	Mean Absolute Error
MSE	$\frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2$	Mean Squared Error
RMSE	$\sqrt{MSE}$	Root Mean Squared Error
MAPE	$\frac{1}{n}\sum_{i=1}^{n}\left\lvert\frac{y_i - \hat{y}_i}{y_i}\right\rvert \times 100$	Mean Absolute Percentage Error
SMAPE	$\frac{1}{n}\sum_{i=1}^{n}\frac{2\lvert y_i - \hat{y}_i\rvert}{\lvert y_i\rvert + \lvert\hat{y}_i\rvert} \times 100$	Symmetric MAPE

13.10 Key Takeaways

1. Time series data is order-dependent and used for forecasting.
2. Understand components: trend, seasonality, cyclicity, noise.
3. Check for stationarity and apply differencing if needed.
4. Use appropriate model (ARIMA, SARIMA, Exponential Smoothing, Prophet, etc.) based on the data.
5. Evaluate models using metrics like MAE, RMSE, MAPE.
6. Try multiple models and compare results.
7. Time series forecasting is widely used in business, finance, retail, weather, and many real-world applications.

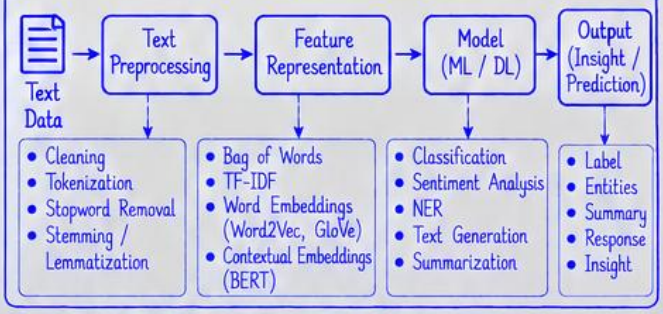
Rahul Yadav

14. Natural Language Processing (NLP) Basics (Text Processing and Applications)

14.1 What is NLP ?

- Natural Language Processing (NLP) is a branch of AI that helps computers understand, interpret and generate human language.
- It combines linguistics, machine learning and deep learning techniques.
- Used in chatbots, search engines, translation, sentiment analysis, summarization, etc.
- NLP works with unstructured text data and converts it into meaningful representations.

14.2 NLP Pipeline (End-to-End Workflow)



14.3 Text Preprocessing Steps

1. **Lowercasing** → Convert text to lowercase.
2. **Remove Special Characters** → Remove symbols, URLs, etc.
3. **Tokenization** → Split text into words (tokens).
4. **Stopword Removal** → Remove common words (e.g., is, the, and).
5. **Stemming** → Reduce word to root form (e.g., running → run).
6. **Lemmatization** → Convert to base word using vocabulary (e.g., better → good).

14.4 Tokenization Example

Text : "I love learning Natural Language Processing!"

Tokenization → ['I', 'love', 'learning', 'Natural', 'Language', 'Processing', '!']

After lowercasing + stopword removal :

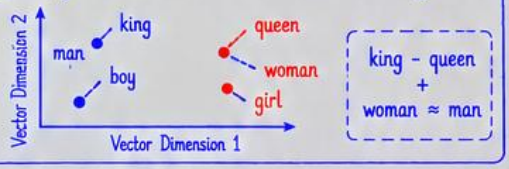
['love', 'learning', 'natural', 'language', 'processing']

14.5 Feature Representation Techniques

Technique	Description	Example
Bag of Words (BoW)	Count of word occurrences	[0, 1, 2, ...]
TF-IDF	Important words with weights	[0.2, 0.0, 1.3, ...]
Word2Vec	Dense vector representation of words	[0.1, -0.3, ...]
GloVe	Pre-trained word embeddings	[0.2, 0.5, ...]
BERT (Transformer)	Contextual embeddings (understands context)	[0.7, -0.1, ...]

14.6 Word Embeddings (Word2Vec)

- Converts words into dense vectors.
- Captures semantic meaning (similar words have similar vectors).
- Two models :
  1. CBOW → Predicts target word from context words.
  2. Skip-gram → Predicts context words from target word.



14.7 Transformer and BERT

- Transformers use self-attention to understand context in a sentence.
- BERT (Bidirectional Encoder Representations from Transformers) learns context from both left and right words.
- Produces contextual embeddings (different meaning for same word in different sentences).

"Bank can mean a financial institution or a river side."

- Bank (financial) → [0.3, -0.1, 0.8, ...]
- Bank (river) → [-0.2, 0.4, 0.1, ...]

14.8 Common NLP Tasks

Task	Description	Example
Text Classification	Assign category to text	Spam or Not Spam
Sentiment Analysis	Detect sentiment (positive, negative, neutral)	"I love this!" → Positive
Named Entity Recognition (NER)	Identify entities (person, location, organization)	"Rahul Yadav works at AU Bank" → [Rahul Yadav (Person), AU Bank (Organization)]
Text Summarization	Generate short summary from long text	Article → Short Summary
Machine Translation	Translate text to another language	"Hello" → "नमस्ते"
Text Generation	Generate human-like text	ChatGPT, conversational AI

14.9 Real-world Applications

1. Chatbots and Virtual Assistants (e.g., customer support)
2. Search Engines (query understanding and ranking)
3. Sentiment Analysis (social media, reviews)
4. Document Classification (emails, legal documents)
5. Text Summarization (news, research papers)
6. Language Translation (Google Translate)
7. Recommendation Systems (based on user text data)

14.10 Key Takeaways

1. NLP helps machines understand and process human language.
2. Preprocessing is important to clean and normalize text data.
3. Use appropriate feature representation (BoW, TF-IDF, Embeddings).
4. Transformers (BERT) give better contextual understanding.
5. NLP has many real-world applications across industries.
6. Choose the right model based on the task and data.
7. Practice with real datasets (e.g., IMDB, Reuters, SQuAD, etc.).

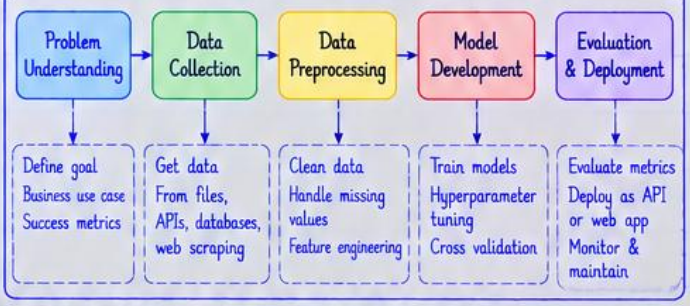
Rahul Yadav

15. Real-world Data Science Projects (End-to-End Implementation)

15.1 Why Real-world Projects are Important ?

- Help in applying theoretical knowledge to real problems.
- Improve problem-solving and analytical skills.
- Give hands-on experience with the complete data science workflow.
- Make you job-ready and portfolio-ready.
- Commonly asked in interviews and help showcase practical skills.

15.2 End-to-End Data Science Project Workflow



15.3 Project 1 – House Price Prediction (Regression)

- **Goal** : Predict house prices based on features like area, bedrooms, location, etc.
- **Dataset** : Kaggle House Prices Dataset
- **Algorithms** : Linear Regression, Random Forest, XGBoost (Regression)
- **Evaluation Metrics** : MAE, RMSE, R^2 Score
- **Tech Stack** : Python, Pandas, Scikit-learn, Matplotlib, Seaborn

15.4 Project 1 – Workflow & Results





Model Comparison

Model	MAE	RMSE	R^2 Score
Linear Regression	0.42	0.62	0.76
Random Forest	0.28	0.39	0.88
XGBoost	**0.26**	**0.37**	**0.89**

15.5 Project 2 – Customer Churn Prediction (Classification)

- **Goal** : Predict whether a customer will churn or not.
- **Dataset** : Telco Customer Churn Dataset
- **Algorithms** : Logistic Regression, Decision Tree, Random Forest, XGBoost
- **Evaluation Metrics** : Accuracy, Precision, Recall, F1-Score, ROC-AUC
- **Tech Stack** : Python, Pandas, Scikit-learn, Seaborn, Matplotlib

15.6 Project 2 – Key Steps & Results



Model Performance

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.78	0.75	0.72	0.73	0.82
Random Forest	0.84	0.82	0.81	0.81	0.88
XGBoost	**0.85**	**0.83**	**0.82**	**0.82**	**0.89**

15.7 Project 3 – Movie Recommendation System (Collaborative Filtering)

- **Goal** : Recommend movies to users based on past ratings.
- **Dataset** : MovieLens Dataset
- **Algorithms** : Collaborative Filtering (User-Based, Item-Based), Matrix Factorization (SVD)
- **Evaluation Metrics** : RMSE, Precision@K, Recall@K.
- **Tech Stack** : Python, Pandas, Surprise Library

15.8 Project 3 – Workflow & Example Recommendations

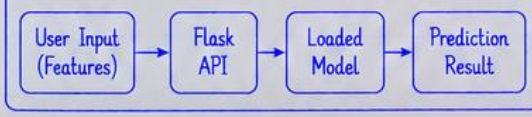


Top 5 Recommended Movies for User

Rank	Movie	Predicted Rating
1	The Dark Knight	4.8
2	Inception	4.7
3	Interstellar	4.6
4	The Prestige	4.5
5	Fight Club	4.4

15.9 Deployment Example (Flask Web App)

- Save the trained model using joblib or pickle.
- Create a Flask/FastAPI app.
- Build a simple UI (HTML/Streamlit) for input.
- Deploy on cloud (Render, Railway, Heroku, AWS).



15.10 Key Takeaways

1. Real-world projects help in applying concepts and building confidence.
2. Understand the problem statement and business context.
3. Perform proper data preprocessing and feature engineering.
4. Try multiple models and compare results using appropriate metrics.
5. Visualize results to get insights.
6. Deploy the model as a web app or API.
7. Add projects to your portfolio (GitHub) for better career opportunities.

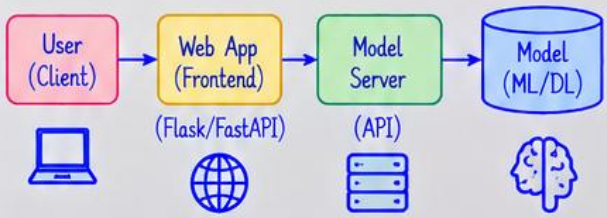
Rahul Yadav

16. Deployment and Productionization (Flask, FastAPI, Docker, Cloud)

16.1 Why Deployment is Important ?

- Makes the model available for real users.
- Bridges the gap between development and real-world usage.
- Ensures scalability, reliability and security.
- Helps in monitoring model performance.
- Supports continuous updates and new features.

16.2 Deployment Architecture (High Level)



16.3 Flask vs FastAPI

Feature	Flask	FastAPI
Type	Micro web framework	Modern, high-performance
Performance	Moderate	High (async support)
Syntax	Simple and easy	Modern and type hints
API Docs	Manual (e.g., Swagger)	Automatic (Swagger UI)
Async Support	Limited	Full async support
Best For	Small projects, learning	Production-grade ML APIs

16.4 Simple FastAPI ML API Example

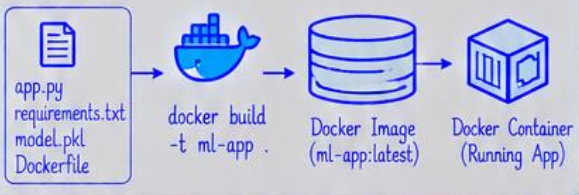
```
from fastapi import FastAPI
import joblib
import numpy as np

app = FastAPI(title="House Price Prediction API")
model = joblib.load("model.pkl")

@app.post("/predict")
def predict(data: dict):
    features = np.array(data["features"]).reshape(1, -1)
    prediction = model.predict(features)[0]
    return {"prediction": float(prediction)}
```

16.5 Dockerizing a Machine Learning App

- Docker packages the application and all dependencies inte a container.
- Ensures same environment in development, testing and production.
- Easy to deploy on cloud platforms.



16.6 Dockerfile Example (FastAPI)

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

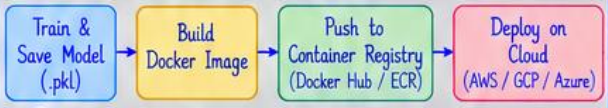
EXPOSE 8000

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

16.7 Deploy on Cloud Platforms

Platform	Features	Use Case
Heroku	Easy deployment, free tier	Small projects, learning
AWS (EC2/ECS)	Scalable, reliable	Production applications
Google Cloud Run	Serverless, auto-scaling	ML APIs with variable traffic
Azure App Service	Easy integration with Azure tools	Enterprise applications

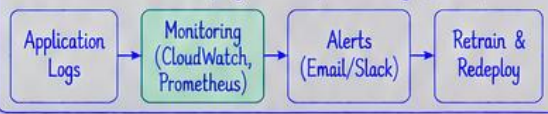
16.8 Deployment Workflow



1. Train the model and save it (e.g., model.pkl).
2. Create a FastAPI/Flask app with the model.
3. Write a Dockerfile and build the image.
4. Push the image to a container registry (Docker Hub, AWS ECR).
5. Deploy on cloud (EC2, ECS, Cloud Run, etc.).
6. Monitor and maintain the application.

16.9 Monitoring and Maintenance

- Monitor API health (uptime, response time).
- Track model performance (accuracy, drift).
- Use logging and error tracking (e.g., ELK, Sentry).
- Set up alerts for failures.
- Retrain and redeploy model periodically (CI/CD).



16.10 Key Takeaways

1. Deployment makes ML models accessible to real users.
2. FastAPI is modern, fast and recommended for production.
3. Docker ensures a consistent and portable environment.
4. Cloud platforms provide scalability and reliability.
5. Always monitor model performance in production.
6. Use CI/CD for regular updates and retraining.
7. Follow best practices for security and maintenance.

Rahul Yadav

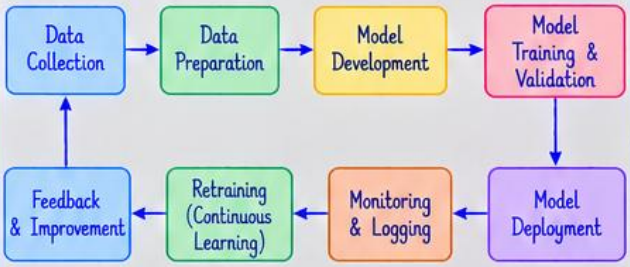
17. MLOps (Machine Learning Operations)

(Build, Deploy, Monitor and Scale ML Models)

17.1 What is MLOps ?

- MLOps (Machine Learning Operations) is a set of practices to automate and streamline the ML lifecycle from development to production.
- Combines Machine Learning, DevOps and Data Engineering.
- Helps in building reliable, scalable and maintainable ML systems.
- Focuses on reproducibility, automation, monitoring and continuous improvement.
- Common in real-world applications like recommendation systems, fraud detection, NLP, etc.

17.2 MLOps Lifecycle



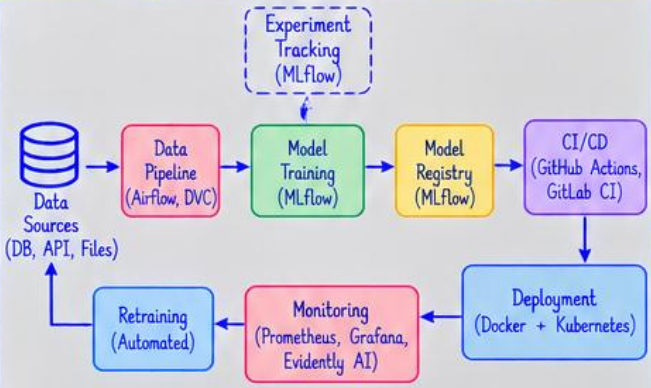
17.3 Key Components of MLOps

Component	Description
Data Pipeline	Automate data collection, cleaning and preprocessing.
Experiment Tracking	Track experiments, parameters and model versions.
Model Registry	Store and manage model artifacts.
CI/CD Pipeline	Automate testing and deployment.
Model Monitoring	Track model performance, data drift, concept drift, and system health.
Retraining Pipeline	Automatically retrain model with new data.
Infrastructure	Use Docker, Kubernetes, Cloud (AWS/GCP/Azure) for scalability.

17.4 Popular MLOps Tools

Tool	Use Case
MLflow	Experiment tracking, model registry, deployment.
DVC (Data Version Control)	Version control for data and models.
Airflow	Workflow orchestration and scheduling.
Kubeflow	End-to-end MLOps on Kubernetes.
Docker	Containerization of ML models.
Kubernetes	Scalable deployment and management.
Prometheus + Grafana	Monitoring and visualization.
Seldon Core / BentoML	Model serving and deployment.
Evidently AI	Data and model monitoring (drift, performance).
GitHub Actions / GitLab CI	CI/CD automation.

17.5 MLOps Architecture (End-to-End)



17.6 Example: MLflow Workflow

1. Log Experiment

```
import mlflow
import mlflow.sklearn

with mlflow.start_run():
    mlflow.log_param("n_estimators", 100)
    mlflow.log_metric("rmse", 0.25)
    mlflow.sklearn.log_model(model, "model")
```

2. Register Model

```
mlflow.register_model(
   "runs:/<run_id>/model", "HousePriceModel"
)
```

3. Load Model for Inference

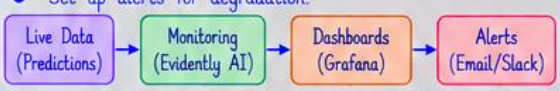
```
model = mlflow.sklearn.load_model(
        "models:/HousePriceModel/1"
)
```

4. Run MLflow UI

```
mlflow ui  # open in browser at http://localhost:5000
```

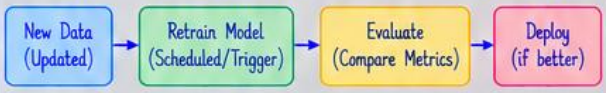
17.7 Model Monitoring

- Monitor prediction quality (accuracy, RMSE, etc.).
- Detect data drift (change in input data distribution).
- Detect concept drift (change in relationship between input and output).
- Monitor system metrics (CPU, memory, latency).
- Use tools like Evidently AI, Prometheus, Grafana.
- Set up alerts for degradation.



17.8 Retraining Strategy

- Retrain model periodically (e.g., weekly/monthly).
- Retrain when data drift or performance drop is detected.
- Use automated pipelines (Airflow/Kubeflow).
- Compare new model with current model.
- If performance is better, deploy new model (canary/blue-green).



17.9 Best Practices

1. Keep data and code versioned (DVC + Git).
2. Use experiment tracking (MLflow).
3. Automate training, testing and deployment (CI/CD).
4. Monitor model and data in production.
5. Use containerization (Docker) and orchestration (Kubernetes).
6. Set up alerts and logging.
7. Document everything and follow reproducible pipelines.

17.10 Key Takeaways

1. MLOps helps in productionizing ML models.
2. Use tools like MLflow, Airflow, Docker, Kubernetes.
3. Monitor model performance and data drift.
4. Automate retraining and deployment.
5. Follow best practices for reliable and scalable ML systems.
6. MLOps is essential for real-world AI/ML applications.
7. Start small with simple pipelines and gradually scale.

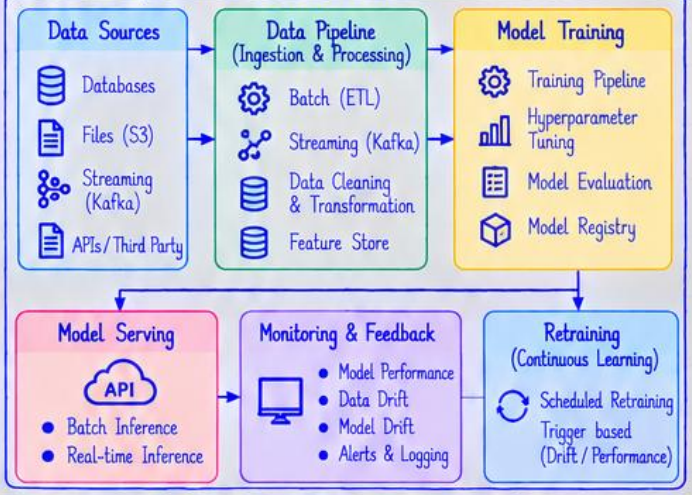
Rahul Yadav

18. System Design for Machine Learning
(End-to-End Architecture and Key Concepts)

18.1 What is ML System Design ?

- ML System Design is the process of designing scalable, reliable and maintainable systems to build, deploy and serve machine learning models in real-world applications.
- It involves data pipeline, model training, model serving, monitoring and continuous improvement.
- Focuses on scalability, low latency, high availability, cost optimization and data governance.
- Similar to software system design, but with additional challenges like data, model versioning, experimentation and drift.

18.2 High Level Architecture



18.3 Batch vs Real-time vs Streaming Inference

Aspect	Batch Inference	Real-time Inference	Streaming Inference
Data Type	Historical data	Single request (real-time)	Continuous data (stream)
Latency	High (minutes/hours)	Low (milliseconds)	Low (milliseconds)
Use Case	Reports, offline predictions	User-facing apps (e.g., recommendation)	Fraud detection, real-time analytics
Processing	Scheduled jobs	REST/gRPC API	Stream processing (Kafka, Spark)
Tools	Airflow, Spark	FastAPI, Flask, TensorFlow Serving	Kafka, Spark Streaming, Flink

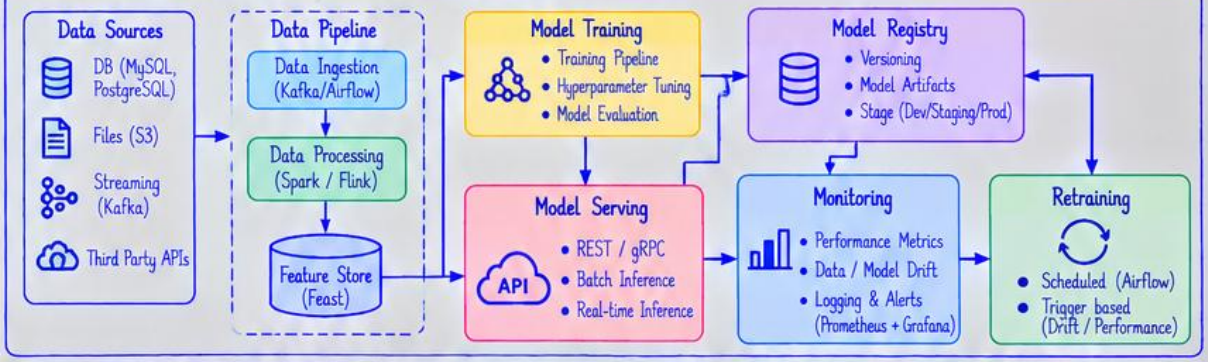
18.4 Model Serving Options

- REST API (FastAPI/Flask) – Simple and widely used.
- TensorFlow Serving – For TensorFlow models.
- TorchServe – For PyTorch models.
- ONNX Runtime – Framework agnostic.
- Batch Inference – For large volume data.
- Serverless (AWS Lambda, GCP Cloud Run) – For small and event-driven workloads.
- Containerized Deployment (Docker + Kubernetes) – For scalable and production-grade serving.

18.5 Feature Store

- Stores precomputed features for training and inference.
- Ensures feature consistency and reuse.
- Reduces training-serving skew.
- Popular tools: Feast, AWS SageMaker Feature Store, Databricks Feature Store.

18.6 Detailed Architecture with Components



18.7 Scalability and Reliability Considerations

- Use stateless model servers and auto-scaling.
- Use load balancer (Nginx / ALB / API Gateway).
- Use caching (Redis) for frequent predictions.
- Implement retry and circuit breaker.
- Use asynchronous processing for batch workloads.
- Ensure high availability (multi-AZ deployment).
- Monitor system metrics (CPU, memory, latency).
- Use canary or blue-green deployment for new models.

18.8 Challenges in ML System Design

1. Handling large and diverse data.
2. Data quality and missing values.
3. Model drift and performance degradation.
4. Real-time low latency serving.
5. Scalability and cost optimization.
6. Continuous retraining and model versioning.

18.9 Real-world Examples

Domain	Use Case	Key Components
E-commerce	Product Recommendation	User data, embedding model, real-time API
Finance	Fraud Detection	Streaming data, real-time inference, monitoring
Healthcare	Disease Prediction	Batch inference, periodic retraining
Media	Content Recommendation	Feature store, real-time serving
Retail	Demand Forecasting	Time series model, batch pipeline

18.10 Key Takeaways

1. ML system design combines data, ML and DevOps.
2. Design scalable and reliable end-to-end pipelines.
3. Use appropriate serving strategy (batch / real-time).
4. Monitor data and model performance continuously.
5. Implement retraining and versioning for model lifecycle.
6. Follow best practices for deployment, security and scalability.

Rahul Yadav

19. Monitoring, Logging and Observability (for ML Systems in Production)

19.1 Why Monitoring is Important ?

- Helps to keep ML models and applications healthy in production.
- Detects issues early (data drift, model drift, errors, high latency).
- Improves reliability, performance and user experience.
- Helps in quick debugging and incident resolution.
- Provides insights for continuous improvement.
- Required for business critical ML applications (e.g., fraud detection, recommendations, etc.).

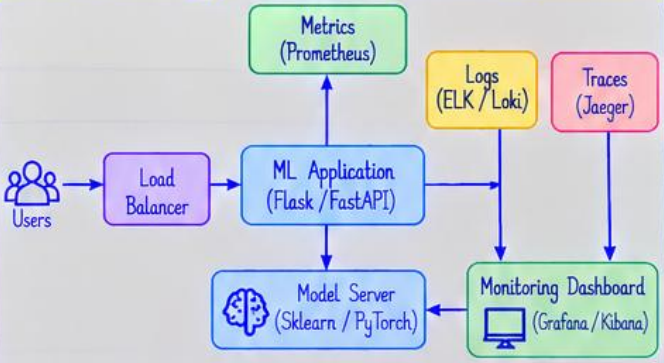
19.2 Observability Pillars

Metrics (What is happening?)
- CPU, Memory, Latency
- Request count
- Prediction latency
- Model accuracy
- Error rate

Logs (What happened?)
- Application logs
- Inference logs
- Error logs
- Data pipeline logs
- Model version logs

Traces (Why did it happen?)
- Request flow
- Service dependencies
- Latency across services
- Identify bottlenecks
- Distributed tracing

19.3 Monitoring Architecture (End-to-End)



19.4 Popular Tools

Category	Tools	Use Case
Metrics	Prometheus, Grafana	Collect and visualize metrics
Logging	ELK Stack (Elasticsearch, Logstash, Kibana), Loki	Collect, store and search logs
Tracing	Jaeger, Zipkin	Distributed tracing
Model Monitoring	Evidently AI, WhyLabs, Fiddler	Data drift, model drift, quality monitoring
Alerting	Alertmanager, Grafana Alerts, Slack	Send alerts on anomalies
Infrastructure	Docker, Kubernetes, CloudWatch	System metrics and infrastructure logs

19.5 Metrics to Monitor for ML Systems

Type	Metric	Description
Application Metrics	Request count	Total API requests
	Response latency	Time taken for prediction
	Error rate	5xx / 4xx errors
	Throughput	Requests per second
Model Metrics	Prediction latency	Time for model inference
	Model accuracy	Accuracy on live data (if labels avaliable)
	Data drift	Change in input data distribution
	Prediction drift	Change in model output distribution
	Feature distribution	Monitor important feature stats
System Metrics	CPU / Memory	Resource usage of server
	Disk usage	Storage utilization
	GPU usage	For deep learning models
	Pod health	Kubernetes pod status

19.6 Example: Prometheus + Grafana (Metrics)

```
# prometetheus.yml (scrape configuration)
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'ml-app'
    static_configs:
      - targets: ['ml-app:8000"]

# In FastAPI application
from prometheus_fastapi_instrumentator import Instrumentator
app = FastAPI()
Instrumentator().instrument(app).expose(app)
```

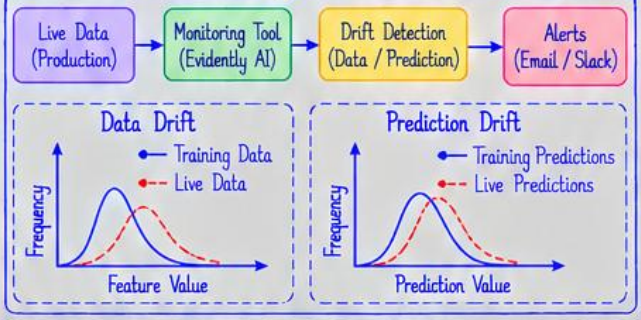
19.7 Example: Logging with ELK (Python)

```
import logging
from pythonjsonlogger import jsonlogger

logger = logging.getLogger("ml_app")
handler = logging.StreamHandler()
formatter = jsonlogger.JsonFormatter()
handler.setFormatter(formatter)
logger.addHandler(handler)
logger.setLevel(logging.INFO)

logger.info("Prediction request received", extra={
    "user_id": "123",
    "model_version": "v1.0",
    "input_size": 10
})
```

19.8 Model Monitoring (Data & Prediction Drift)



19.9 Alerting and Incident Management



19.10 Key Takeaways

1. Monitoring, logging and tracing are essential for ML in production.
2. Use metrics, logs and traces to get complete observability.
3. Monitor both system metrics and model-specific metrics.
4. Detect data drift and prediction drift using tools like Evidently AI.
5. Set up alerts for critical issues (latency, errors, drift, resource usage).
6. Use dashboards (Grafana/Kibana) for real-time insights.
7. Follow best practices for reliable and maintainable ML systems.

Rahul Yadav.